Real-world (persist and read back real data) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

A SQLite connection doesn't save anything to disk the instant you `execute()` an `INSERT` — it stages the change, and `commit()` is what actually writes it. Forgetting `commit()` is a real, common bug: your code runs with no errors, but the data silently isn't there the next time you look.

`CREATE TABLE IF NOT EXISTS` matters because your FastAPI app re-runs this statement every time it starts — without `IF NOT EXISTS`, the second startup would crash trying to create a table that already exists.

By default, `fetchall()` gives you plain tuples — `(1, "hi", "You said: hi", "2026-...")` — with no field names. A client can't use that without guessing the column order. `conn.row_factory = sqlite3.Row` makes each row behave like a dict, and `dict(row)` turns it into a real one.

The exact pieces:

- `conn = sqlite3.connect("runs.db", check_same_thread=False)` — FastAPI can call your route from a different thread than the one that opened the connection, so this flag is needed.
- `conn.row_factory = sqlite3.Row` — set once, right after connecting.
- `"CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY AUTOINCREMENT, input TEXT, output TEXT, created_at TEXT)"` — `AUTOINCREMENT` gives each row a unique, growing `id` for free.
- **Always use `?` placeholders, never f-strings, for values going into SQL:** `conn.execute("INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)", (input_text, output_text, timestamp))` — this is what prevents SQL injection.
- `datetime.now(timezone.utc).isoformat()` for a real, sortable timestamp string.

- A plain `for` loop that appends `dict(row)` for each row to a list — safe to return straight from a route.
- A small helper file (`practice/runs_db.py`) holding the connect/insert/select functions — the same split as the Build Task's `db/database.py` .
- `GET /runs/{run_id}` : `SELECT * FROM runs WHERE id = ?` with `(run_id,)` , then `.fetchone()` — it returns `None` when no row matches, so raise `HTTPException(404, ...)` in that case.
- A `RunRecord(BaseModel)` with `id, input, output, created_at` , used as `response_model` — so every returned row's shape is checked, same as `ChatResponse` .

Hint 2 — The plan, and almost the whole thing

```
same setup, plus:
    conn.row_factory = sqlite3.Row    # rows behave like dicts now

define RunRecord(BaseModel): id, input, output, created_at

route GET /runs, response_model=list[RunRecord]:
    select * from runs
    loop over the rows, append dict(row) to a list, return the list

route GET /runs/{run_id}, response_model=RunRecord:
    select the one row matching run_id
    if it doesn't exist: raise a 404
    otherwise return dict(row)
```

Here's most of it — fill in the single-row lookup yourself:

```
# practice/sqlite_persistence_practice.py — Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
```

```
    )
""")

class ChatRequest(BaseModel):
    message: str

class RunRecord(BaseModel):
    id: int
    input: str
    output: str
    created_at: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs", response_model=list[RunRecord])
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    result = []
    for row in rows:
        result.append(dict(row))
    return result

@app.get("/runs/{run_id}", response_model=RunRecord)
def get_run(run_id: int):
    # your turn: SELECT * FROM runs WHERE id = ?, with (run_id,) as params
    # if fetchone() gives back None, raise HTTPException(404, ...)
    ...
```

Test the 404 path on purpose: request an `id` you know doesn't exist and confirm you get a clean 404, not a 500 or a crash.

Full solution: Show me the solution